

Caso de Estudio: Conteo de Palabras con Hadoop MapReduce

Información del Proyecto

Maestría: Ciencia de Datos

Universidad: Pontificia Universidad Javeriana

Materia: Gestión de Datos

Estudiantes: Edwin Silva Salas, Carlos Preciado Cárdenas, Cristian Restrepo Zapata

Fecha de inicio: Diciembre 2025



Introducción

El análisis de frecuencia de palabras es una técnica fundamental en el procesamiento de lenguaje natural y minería de textos, utilizada para identificar patrones, extraer información relevante y comprender la estructura de documentos textuales. En el contexto de Big Data, donde los volúmenes de información crecen exponencialmente, se requieren herramientas capaces de procesar grandes cantidades de datos de manera eficiente y escalable.

Apache Hadoop MapReduce representa una solución robusta para el procesamiento distribuido de datos masivos, permitiendo dividir tareas computacionales complejas en operaciones más simples que pueden ejecutarse en paralelo sobre múltiples nodos. El paradigma MapReduce, inspirado en las funciones map y reduce de la programación funcional, ofrece un modelo de programación simple pero poderoso para el análisis de grandes conjuntos de datos.

Este trabajo presenta la implementación técnica de un sistema de conteo y análisis de frecuencia de palabras aplicado a un documento académico sobre inteligencia artificial en el sector bancario. La solución desarrollada integra múltiples tecnologías: extracción de texto desde formato PDF, almacenamiento en HDFS (Hadoop Distributed File System), y procesamiento mediante MapReduce con Python.

Entorno de Trabajo

- **Sistema Operativo:** Ubuntu 22.04 LTS en WSL (instalado WSL)
- **Hadoop:** Versión 3.3.6
- **Python:** Versión 3.x
- **Documento Origen:** Inteligencia_Rodriguez_ICE_2022.pdf

Paso 1: Conversión de PDF a Texto Plano

Descripción

El primer paso del proceso consiste en extraer el contenido textual del documento PDF académico "Inteligencia Artificial en el Sector Bancario" y guardarlo en un archivo de texto plano que pueda ser procesado por Hadoop.

Herramienta Utilizada

Se utiliza la herramienta **pdftotext** del paquete **poppler-utils**, que permite extraer texto de archivos PDF manteniendo la estructura del contenido.

Implementación

Se creó un script de Python llamado **pdf_to_txt.py** ubicado en la carpeta **/code/**:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Conversión de PDF a Texto Plano
Taller: Caso Conteo de Palabras con Hadoop
"""

import subprocess
import os

# Rutas de archivos
PDF_FILE = "/home/esilvas/Documents/hadoop-python/files/Inteligencia_Rodriguez_ICE_2022.pdf"
OUTPUT_FILE = "/home/esilvas/Documents/hadoop-python/files/texto_plano.txt"

def pdf_to_text():
    """
    Convierte un archivo PDF a texto plano usando pdftotext
    """
    try:
        print(f"Convirtiendo PDF a texto plano...")
        print(f"Archivo origen: {PDF_FILE}")
        print(f"Archivo destino: {OUTPUT_FILE}")

        # Ejecutar pdftotext para extraer el texto
        subprocess.run([
            'pdftotext',
            PDF_FILE,
            OUTPUT_FILE
        ], check=True)

        # Verificar que se creó el archivo
        if os.path.exists(OUTPUT_FILE):
            # Obtener tamaño del archivo
            size = os.path.getsize(OUTPUT_FILE)
            print(f"\nConversión exitosa!")
            print(f"Archivo creado: {OUTPUT_FILE}")
            print(f"Tamaño: {size} bytes")

            # Contar líneas y palabras
            with open(OUTPUT_FILE, 'r', encoding='utf-8') as f:
                lines = f.readlines()
                words = sum(len(line.split()) for line in lines)
```

```
        print(f"Líneas: {len(lines)}")
        print(f"Palabras aproximadas: {words}")
    else:
        print("Error: No se pudo crear el archivo de salida")

except subprocess.CalledProcessError as e:
    print(f"Error al ejecutar pdftotext: {e}")
except Exception as e:
    print(f"Error: {e}")

if __name__ == "__main__":
    pdf_to_text()
```

Ejecución del Script

Para ejecutar el script, se utilizó el siguiente comando desde la terminal:

```
cd /home/esilvas/Documents/hadoop-python/code
python3 pdf_to_txt.py
```

Resultado de la Conversión

```
Convirtiendo PDF a texto plano...
Archivo origen: /home/esilvas/Documents/hadoop-
python/files/Inteligencia_Rodriguez_ICE_2022.pdf
Archivo destino: /home/esilvas/Documents/hadoop-
python/files/texto_plano.txt

Conversión exitosa!
Archivo creado: /home/esilvas/Documents/hadoop-
python/files/texto_plano.txt
Tamaño: 66525 bytes
Líneas: 1116
Palabras aproximadas: 9574
```

Estructura del Archivo Resultante

El archivo `texto_plano.txt` generado contiene:

- **Tamaño:** 66,525 bytes (≈ 65 KB)
- **Líneas:** 1,116
- **Palabras:** Aproximadamente 9,574

Muestra del Contenido (Primeras líneas):

Teresa Rodríguez de las Heras Ballell*

INTELIGENCIA ARTIFICIAL EN EL SECTOR
BANCARIO: REFLEXIONES SOBRE SU RÉGIMEN
JURÍDICO EN LA UNIÓN EUROPEA

El objetivo de este trabajo es aproximarnos al estado actual del régimen jurídico aplicable en la Unión Europea al uso de sistemas de inteligencia artificial en el sector financiero, y reflexionar sobre la necesidad de formular principios y reglas que aseguren una automatización responsable de los procesos de toma de decisiones y que sirvan de guía para implementar soluciones de inteligencia artificial en la actividad bancaria.

Paso 2: Carga de Datos a HDFS

2.1 Descripción

En este paso, el archivo de texto plano extraído del PDF ([texto_plano.txt](#)) se carga en el sistema de archivos distribuido de Hadoop (HDFS). Esto permite que los datos estén disponibles para ser procesados de manera distribuida por los nodos de Hadoop.

2.2 Comandos utilizados

```
# Crear el directorio de entrada en HDFS (si no existe)
hdfs dfs -mkdir -p /user/esilvas/wordcount/input

# Subir el archivo de texto a HDFS
hdfs dfs -put /home/esilvas/Documents/hadoop-python/files/texto_plano.txt
/user/esilvas/wordcount/input/

# Verificar que el archivo está en HDFS
hdfs dfs -ls /user/esilvas/wordcount/input/
```

2.3 Importancia

Cargar los datos a HDFS es fundamental para aprovechar el procesamiento distribuido de Hadoop. HDFS permite que los datos sean accesibles por todos los nodos del clúster, facilitando la ejecución eficiente de tareas MapReduce sobre grandes volúmenes de información.

2.4 Automatización con Python

Se creó el script [carga_datos_HDFS.py](#) para automatizar este proceso desde Python, facilitando la integración y repetibilidad del flujo de trabajo.

Paso 3: Implementación MapReduce

[Pendiente - Se documentará posteriormente]

Paso 4: Ejecución y Resultados

[Pendiente - Se documentará posteriormente]

Conclusiones

[Se completará al finalizar todos los pasos]

Referencias

- Apache Hadoop Documentation: <https://hadoop.apache.org/docs/r3.3.6/>
- Poppler Utils: <https://poppler.freedesktop.org/>
- Python Subprocess Documentation: <https://docs.python.org/3/library/subprocess.html>